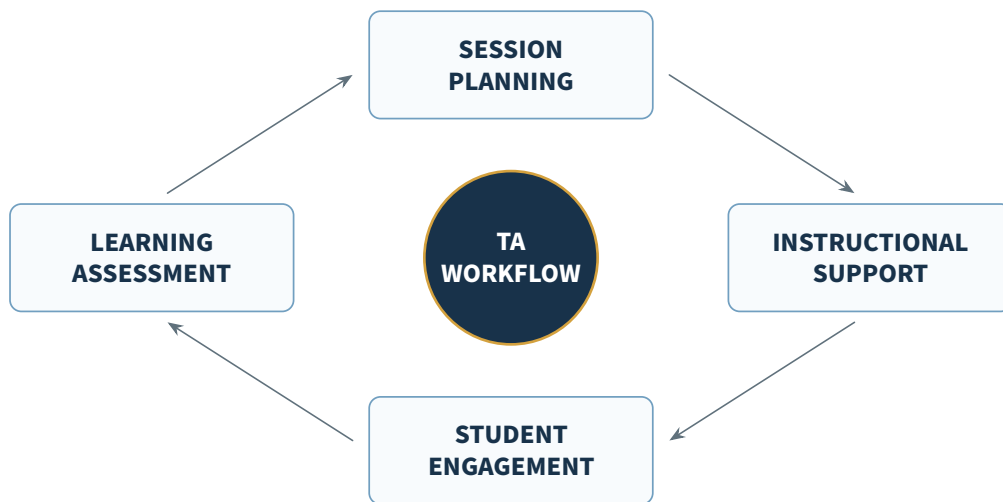




Introduction to Data Science

Teaching Assistant Guide



COURSE OUTLINE

Concise, lab-aligned notes on R programming, data structures, data management, visualization, correlation, transformation, and scaling.

PREPARED BY

Reeyan Afzal Khan

Graduate Research Assistant

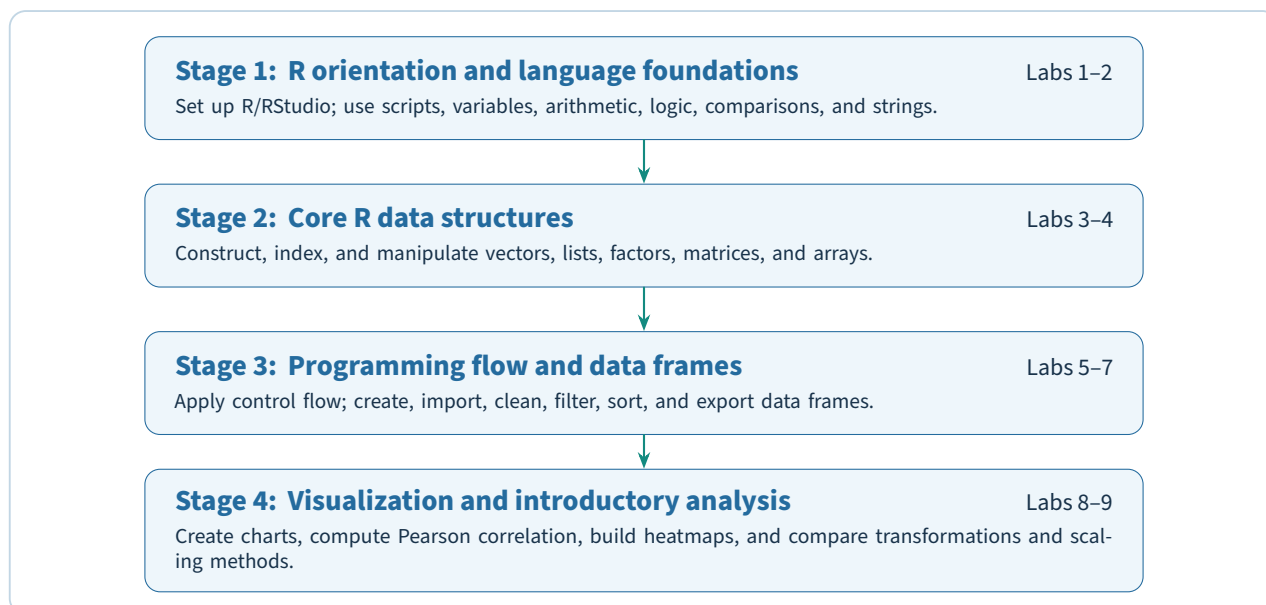


Learning Path

Course Map: From R Commands to Data Insight	ii
Stage 1 R Orientation and Language Foundations	1
Chapter 1 R and RStudio Fundamentals	2
Lab 1 of 9 — R setup, sequences, rounding, and user input	6
Chapter 2 Logic, Comparisons, and Strings	7
Lab 2 of 9 — BMI conditions and string manipulation	11
Stage 2 Core R Data Structures	12
Chapter 3 Vectors and Lists	13
Lab 3 of 9 — Exam records with vectors and lists	17
Chapter 4 Factors, Matrices, and Arrays	19
Lab 4 of 9 — Factors, matrix operations, and multidimensional arrays	23
Stage 3 Programming Flow and Data Frames	25
Chapter 5 Control Flow	26
Lab 5 of 9 — Leap years, cubes, and Armstrong numbers	30
Chapter 6 Data Frame Fundamentals	32
Lab 6 of 9 — Build and extend a student data frame	36
Chapter 7 Managing Data Frames	37
Lab 7 of 9 — Clean, filter, and report on external datasets	41
Stage 4 Visualization and Introductory Analysis	43
Chapter 8 Data Visualization in R	44
Lab 8 of 9 — Turn data-management results into visual evidence	49
Chapter 9 Correlation, Transformation, and Scaling	51
Lab 9 of 9 — Correlation heatmaps and normalization comparison	55
Stage 5 Capstone	57
Capstone: Cleaning a Real Messy Dataset	58

Course Map: From R Commands to Data Insight

The course progresses from R fundamentals to data structures, data management, visualization, and introductory analysis.



Key idea

The sequence is cumulative: Labs 7–9 assume proficiency with indexing, conditions, data frames, packages, and files.

Lab	Main topic	Practical emphasis
1	RStudio, R, variables, arithmetic	Set up R; write code; use assignment, types, arithmetic, and input.
2	Logic, relations, strings	Evaluate Boolean conditions and manipulate text.
3	Vectors and lists	Store, index, and manipulate homogeneous and mixed collections.
4	Factors, matrices, arrays	Work with categorical values and multidimensional structures.
5	Selection and repetition	Select code paths and repeat operations.
6	Data frames	Create, inspect, access, and modify tabular data.
7	Managing data frames	Import, clean, filter, sort, and export data.
8	Data visualization	Create and interpret common statistical charts.
9	Correlation and scaling	Analyze relationships and transform numeric data.

Stage 1

R Orientation and Language Foundations

R and RStudio Fundamentals

Chapter at a glance

Before you start: No prior R programming is required.

By the end, you can: set up R/RStudio; use scripts, variables, assignment, object classes, arithmetic, and keyboard input.

1.1 R and RStudio setup

Install **R** first, then **RStudio Desktop**. Verify the setup in the Console:

```
1 + 2
```

The result 3 confirms that RStudio can use the R runtime.

Key idea

R executes the code; RStudio provides the development environment.

1.1.1 Console versus Script

Use the Console for immediate evaluation and scripts for saved, reproducible work.

Area	What the lab uses it for	Example
R Console	Immediate commands and quick tests.	2 + 3
R Script	Saved statements that can be edited and rerun.	test.R

Worked example

The first saved script in the handout is:

```
# My first program in R Programming
myString <- "Hello, World!"
print(myString)
```

begins a comment, <- assigns a value, and print() displays it.

1.2 Packages

Install a package once, then load it in each session that uses it:

```
install.packages("package_name")
library(package_name)
```

`install.packages()` installs; `library()` loads.

1.3 Variables and object names

Create variables and inspect the workspace:

```
var.1 = 5
var_1 = 7
x = 1
print(ls())
print(ls(pattern = "var"))
```

`ls()` lists workspace objects; its `pattern` argument filters their names. R object names may contain periods or underscores.

1.4 Assignment operators

R supports assignment in either direction:

Operator	Handout description
<code><-</code> , <code><=<</code> , <code>=</code>	Leftwards assignment
<code>-></code> , <code>->></code>	Rightwards assignment

The examples combine assignment with several value types:

```
Gender <- "Female"
height <=< 152
Weight = 81
3 -> f
23.5 ->> x
b <- seq(from = 0, to = 5)
c <- seq(from = 0, to = 10, by = 2)
v = 2L
w <=< 2 + 5i
a <- charToRaw("Hello")
```

`seq()` generates sequences; `2L` is an integer; `2 + 5i` is complex; and `charToRaw()` returns raw byte values.

Technical note

`<=<` and `->>` assign outside the current environment. Prefer `<-` for ordinary work because nonlocal assignment can make code difficult to trace.

1.4.1 Inspecting an object's class

The handout checks object classes with `class()`:

```
print(class(Gender))
print(class(height))
print(class(f))
print(class(x))
print(class(b))
print(class(v))
print(class(w))
```

Key idea

Use `class()` to inspect an object's class instead of inferring it from the name.

1.5 Arithmetic operations

R provides the following arithmetic operators:

Operator	Operation
+	Addition
-	Subtraction
*	Multiplication
** or ^	Exponentiation
/	Division
%%	Modulus / remainder
%%/	Integer division

The examples combine these operators in a BMI calculation:

```
print(f + 3)
print(height - x)
print(Weight * 2)
print(b ** 2)
print(c ^ 5)

m = height / 100
print(Weight / (m ** 2))
BMI = Weight / (m ** 2)

print(b %% 2)
print(c %/% 2)
```

The BMI calculation reuses the height in meters and stores the result.

1.6 Reading user input

`readline()` returns text, so numeric input requires conversion:

```
name <- readline(prompt = "Enter name: ")
age <- readline(prompt = "Enter age: ")

# convert character into numeric
age <- as.numeric(age)

print(paste("Hi,", name,
            "this year you are", age, "years old."))
```

This is a basic input–process–output pattern.

1.7 Built-in help and demonstrations

Use built-in help and demonstrations to explore functions:

```
?paste
demo(graphics)
demo(image)
```

Check your understanding

1. Why does the handout install R before RStudio?
2. What is the practical difference between testing one expression in the Console and saving several statements in an R Script?
3. What does `ls(pattern = "var")` do differently from `ls()`?
4. Which operators in the handout return a remainder and an integer quotient?
5. Why does the user-input example call `as.numeric(age)`?

Chapter recap

- Use the Console for quick tests and scripts for reproducible programs.
- Prefer `<-` for routine assignment; inspect objects with `class()` and `ls()`.
- Arithmetic and input conversion support simple input–process–output programs.

PRACTICAL MILESTONE

Lab 1 of 9

R setup, sequences, rounding, and user input

Lab 1b guided practice

Coverage. Chapter 1.

Goal. Apply basic sequence generation, arithmetic, rounding, and keyboard input.

Lab guidance

Complete the three tasks using only the functions introduced in Chapter 1.

Task 1 — Create 20 numbers and display their squares.

```
numbers <- 1:20
squares <- numbers ^ 2

print(numbers)
print(squares)
```

A compact two-column display is:

```
print(cbind(numbers, squares))
```

Task 2 — Round two given values.

```
num1 <- 0.956786
num2 <- 7.8345901

print(round(num1, 2))
print(round(num2, 3))
```

The expected rounded values are 0.96 and 7.835.

Task 3 — Read a circle radius and calculate area.

```
radius <- readline(prompt = "Enter radius: ")
radius <- as.numeric(radius)

area <- pi * radius ^ 2
print(paste("Circle area:", area))
```

The program reads, converts, calculates, and displays.

Lab deliverable

Submit all three programs and verified outputs to your instructor.

Logic, Comparisons, and Strings

Chapter at a glance

Before you start: You should be able to create variables and print results.

By the end, you can: use logical and relational operators; create valid strings; and concatenate, format, measure, transform, and extract text.

2.1 Logical operations

The core logical operators are:

Operator	Meaning in the handout
!	Logical NOT
&	Logical AND
	Logical OR

With single Boolean values:

```
a = TRUE
b = FALSE

print(a & b)
print(a & !b)
print(a | b)
print(!a | b)
```

The following code compares element-wise and short-circuit forms:

```
c = c(TRUE, FALSE)
d = c(FALSE, FALSE)

print(c && d)
print(c || d)

v <- c(3, 1.5, TRUE)
t <- c(4, 2.1, FALSE)

print(!v)
print(!t)
print(v & t)
print(v | t)
print(v && t)
print(v || t)
```

Key idea

& and | operate element by element. && and || inspect only the first element and may skip evaluation of the second expression. Use the double-character forms for scalar control-flow conditions.

2.2 Relational operations

Relational operators compare values and return logical results.

Operator	Meaning
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to
==	Equal to
!=	Not equal to

```
x = 5
y = 9
z = 2 + 3

print(x > y)
print(x < y)
print(x <= z)
print(y >= z)
print(x == z)
print(x != y)
```

Because z evaluates to 5, `x == z` is TRUE.

Comparisons also operate element by element on vectors:

```
v <- c(2, 5.5, 6, 9)
t <- c(8, 2.5, 14, 9)

print(v > t)
print(v < t)
print(v <= t)
print(v >= t)
print(v == t)
print(v != t)
```

2.3 Constructing strings

R strings may use single or double quotation marks:

```
a <- 'Start and end with single quote'
b <- "Start and end with double quotes"
c <- "single quote ' in between double quotes"
d <- 'Double quotes " in between single quote'
```

Embedded delimiters of the same type must be escaped.

Technical note

Unmatched or unescaped delimiters cause syntax errors. Keep quotation marks balanced.

2.4 Escape characters

Common escape sequences include:

Sequence	Description
\"	Double quote
\\	Backslash
\n	New line
\r	Carriage return
\t	Tab

```
str <- "We are the so-called \"Vikings\", from the north."
cat(str)

str <- "We are the so-called \\Vikings\\, from the north."
cat(str)

str <- "We are the \rso-called Vikings\nfrom the north."
cat(str)

str <- "\tWe are the so-called Vikings from the north."
cat(str)
```

`cat()` renders the escape sequences instead of printing a quoted representation.

2.5 Concatenating strings

`paste()` combines text with configurable separators:

```
a <- "Hello"
b <- "How"
c <- "are you? "

print(paste(a, b, c))
print(paste(a, b, c, sep = "-"))
print(paste(a, b, c, sep = "", collapse = ""))
```

`sep` joins arguments within each result; `collapse` joins multiple result elements.

2.6 Formatting numbers and strings

`format()` controls display without changing the underlying numeric value:

```
# Total number of displayed digits; final digit is rounded.
result <- format(23.123456789, digits = 9)
print(result)

# Scientific notation.
result <- format(c(6, 13.14521), scientific = TRUE)
print(result)

# Minimum digits to the right of decimal point.
result <- format(23.47, nsmall = 5)
print(result)

# format() returns formatted text.
result <- format(6)
print(result)

# Width and justification.
result <- format(13.7, width = 6)
print(result)
result <- format("Hello", width = 8, justify = "l")
print(result)
result <- format("Hello", width = 8, justify = "c")
```

```
print(result)
```

2.7 Measuring and changing strings

Use `nchar()` in base R or `str_length()` from `stringr` to count characters:

```
str <- "Count the number of characters"
print(nchar(str))

install.packages("stringr")
library(stringr)
str_length(str)
```

Case conversion is demonstrated with:

```
print(toupper("Changing To Upper"))
print(tolower("Changing To Lower"))
```

Case conversion supports case-insensitive comparison and standardized output.

2.8 Extracting parts of a string

`substring()` and `substr()` extract character ranges:

```
# 5th through 7th characters
result <- substring("Extract", 5, 7)
print(result)

# first character
result <- substr("Learn Code Tech", 1, 1)
print(result)
```

Both functions use starting and ending positions.

Check your understanding

1. What type of value is returned by a relational comparison such as `x < y`?
2. Why does `x == z` evaluate as true in the handout example?
3. What is the role of a backslash in `"Vikings"` inside a double-quoted string?
4. Which function in the handout converts text to uppercase?
5. Which functions are used to extract character ranges from a string?

Chapter recap

- Use element-wise logical operators for vectors and short-circuit forms for scalar conditions.
- Strings require balanced delimiters; escape sequences represent special characters.
- Base R provides functions for joining, formatting, measuring, changing, and extracting text.

PRACTICAL MILESTONE

Lab 2 of 9

BMI conditions and string manipulation

Lab 2b guided practice

Coverage. Chapter 2.

Goal. Apply logical expressions and string functions to three input tasks.

Task 1 — BMI category flags.

Use continuous boundaries so every valid BMI belongs to one category. The Boolean flags avoid selection statements, which are introduced in Lab 5.

```
weight <- as.numeric(readline("Enter weight (kg): "))
height <- as.numeric(readline("Enter height (m): "))

bmi <- weight / (height ^ 2)

underweight <- bmi < 18.5
normal <- bmi >= 18.5 & bmi < 25
overweight <- bmi >= 25 & bmi < 40
obese <- bmi >= 40.0

print(paste("BMI:", bmi))
print(paste("Underweight:", underweight))
print(paste("Normal:", normal))
print(paste("Overweight:", overweight))
print(paste("Obese:", obese))
```

Task 2 — Case-insensitive comparison of two strings.

```
string1 <- readline("Enter string 1: ")
string2 <- readline("Enter string 2: ")

same <- toupper(string1) == toupper(string2)
print(paste("Both inputs are similar:", same))
```

Converting both inputs to one case makes the comparison case-insensitive.

Task 3 — Uppercase a name and partially display a phone number.

```
name <- readline("Enter name: ")
phone <- readline("Enter phone number: ")

name_upper <- toupper(name)
first3 <- substr(phone, 1, 3)
last4 <- substr(phone, nchar(phone) - 3, nchar(phone))

print(paste("Hi,", name_upper,
            ". A verification code has been sent to",
            first3, "-xxxxx", last4))
```

The output retains the first three and last four digits while masking the middle.

Lab deliverable

Submit all three programs and demonstrate a successful case-insensitive comparison.

Stage 2

Core R Data Structures

Vectors and Lists

Chapter at a glance

Before you start: You should be able to create variables, call functions, and use logical operators.

By the end, you can: construct, index, sort, and calculate with vectors; and create, name, access, and modify lists.

3.1 R vectors

A vector stores values of a common type and can be created with `:`, `seq()`, or `c()`.

3.1.1 Constructing vectors

```
# Integer-like sequence
v <- 5:13
print(v)

# Sequence beginning at a decimal value
v <- 6.6:12.6
print(v)

# End value does not fall exactly on the sequence
v <- 3.8:11.4
print(v)

# Explicit step size
print(seq(5, 9, by = 0.4))
```

Mixing character, numeric, and logical values triggers type coercion:

```
s <- c('apple', 'red', 5, TRUE)
print(s)
```

Key idea

All elements in the mixed vector become character values. Use a list when components must retain different types.

3.1.2 Accessing vector elements

The days vector demonstrates positive, logical, and negative indexing:

```
t <- c("Sun", "Mon", "Tue", "Wed", "Thurs", "Fri", "Sat")

# Position indexing
u <- t[c(2, 3, 6)]
print(u)

# Logical indexing
v <- t[c(TRUE, FALSE, FALSE, FALSE, FALSE, TRUE, FALSE)]
print(v)

# Negative indexing
```



```
x <- t[c(-2, -5)]
print(x)

# Select the seventh element
y <- t[7]
print(y)
```

Positive indices select positions, logical indices select TRUE positions, and negative indices exclude positions.

Technical note

Numeric 0/1 indexing is not a Boolean mask in R: zeros are ignored and each 1 selects position 1. Use TRUE/FALSE values or explicit positions instead.

3.2 Vector arithmetic

Arithmetic on equal-length vectors is element-wise:

```
v1 <- c(3, 8, 4, 5, 0, 11)
v2 <- c(4, 11, 0, 8, 1, 2)

print(v1 + v2)
print(v1 - v2)
print(v1 * v2)
print(v1 / v2)
```

Division by zero produces a non-finite result such as Inf or NaN.

3.2.1 Vector element recycling

Lab 3a next shortens the second vector:

```
v1 <- c(3, 8, 4, 5, 0, 11)
v2 <- c(4, 11)

# Handout note: v2 behaves as c(4, 11, 4, 11, 4, 11)
print(v1 + v2)
print(v1 - v2)
```

R recycles the shorter vector. A warning occurs when the longer length is not a multiple of the shorter length.

3.3 Sorting vectors

```
v <- c(3, 8, 4, 5, 0, 11, -9, 304)
print(sort(v))
print(sort(v, decreasing = TRUE))

v <- c("Red", "Blue", "yellow", "violet")
print(sort(v))
print(sort(v, decreasing = TRUE))
```

sort() handles numeric and character vectors; decreasing = TRUE reverses the order.

3.4 R lists

A list is introduced using a mixture of strings, numbers, a vector, and a logical value:

```
list_data <- list(
  "Red",
```

```
"Green",
c(21, 32, 11),
TRUE,
51.23,
119.1
)
print(list_data)
```

Unlike a vector, a list preserves heterogeneous component types.

3.4.1 Naming list elements

```
list_data <- list(
  c("Jan", "Feb", "Mar"),
  list("green", 12.3)
)

names(list_data) <- c("1st Quarter", "A Inner list")
print(list_data)
```

Names make list access more readable.

3.4.2 Accessing list elements

Lists support position- and name-based access:

```
list_data <- list(
  c("Jan", "Feb", "Mar"),
  list("green", 12.3)
)
names(list_data) <- c("1st_Quarter", "A_Inner_list")

print(list_data[1])
print(list_data[2])
print(list_data$A_Inner_list)
print(which(list_data$`1st_Quarter` == "Feb"))
```

`which()` returns the position of "Feb" within the named component.

Key idea

Single brackets return a sublist; double brackets or `$` return the component itself.

3.4.3 Manipulating a list

```
list_data <- list(
  c("Jan", "Feb", "Mar"),
  list("green", 12.3)
)
names(list_data) <- c("1st_Quarter", "A_Inner_list")

# Add a third element
list_data[3] <- "New_element"
print(list_data[3])

# Remove it
list_data[3] <- NULL
print(list_data[3])

# Update the second element
list_data[2] <- "updated_element"
print(list_data[2])
```

Assignment adds or updates an element; assigning `NULL` removes it.

Check your understanding

1. What happens to the numeric and logical values in the handout's mixed character vector?
2. How does negative vector indexing differ from positive position indexing?
3. What repeated form of `v2` is stated in the vector-recycling example?
4. Why is a list suitable for the Lab 3a example containing strings, numbers, a vector, and a logical value?
5. How does `list_data$A_Inner_list` differ in intent from accessing an unnamed position?

Chapter recap

- Vectors use a common type and support positive, logical, and negative indexing.
- Vector arithmetic is element-wise and may recycle shorter operands.
- Lists preserve mixed types and support named access and modification.

PRACTICAL MILESTONE

Lab 3 of 9

Exam records with vectors and lists

Lab 3b guided practice

Coverage. Chapter 3.

Goal. Use vectors for grade analysis and named lists for student records.

Task 1 — Grade distribution and pass/fail values.

```
scores <- c(
  33, 24, 54, 94, 16, 89, 60, 6, 77, 61,
  13, 44, 26, 24, 73, 73, 90, 39, 90, 54
)

grade_A <- sum(scores >= 90 & scores <= 100)
grade_B <- sum(scores >= 80 & scores <= 89)
grade_C <- sum(scores >= 70 & scores <= 79)
grade_D <- sum(scores >= 60 & scores <= 69)
grade_E <- sum(scores >= 50 & scores <= 59)
grade_F <- sum(scores <= 49)

print(c(A = grade_A, B = grade_B, C = grade_C,
        D = grade_D, E = grade_E, F = grade_F))

pass <- scores > 49
print(pass)
```

The counts are A=3, B=1, C=3, D=2, E=2, and F=9.

Task 2 — Named list of students and exam scores.

```
students <- c(
  "Robert", "Hemsworth", "Scarlett", "Evans", "Pratt",
  "Larson", "Holland", "Paul", "Simu", "Renner"
)

exam <- c(59, 71, 83, 68, 65, 57, 62, 92, 92, 59)

records <- list(student_name = students, exam_score = exam)

highest <- max(records$exam_score)
lowest <- min(records$exam_score)
average <- mean(records$exam_score)

print(paste("Highest Score:", highest))
print(paste("Lowest Score:", lowest))
print(paste("Average Score:", average))
print(paste("Student with highest score:",
  paste(records$student_name[records$exam_score == highest],
    collapse = ", ")))
print(paste("Student with lowest score:",
  paste(records$student_name[records$exam_score == lowest],
    collapse = ", ")))
```

Paul and Simu share the highest score (92); Larson has the lowest (57); the mean is 70.8.

Task 3 — Append Chemistry and Physics scores.

```
records$chemistry <- c(59, 71, 83, 68, 65, 57, 62, 92, 92, 59)
records$physics <- c(89, 86, 65, 52, 60, 67, 40, 77, 90, 61)

chem_fail <- sum(records$chemistry <= 49)
phys_fail <- sum(records$physics <= 49)
```

```
chem_best <- max(records$chemistry)
phys_best <- max(records$physics)

print(paste("Chemistry failures:", chem_fail))
print(paste("Physics failures:", phys_fail))
print(records$student_name[records$chemistry == chem_best])
print(records$student_name[records$physics == phys_best])
```

No student fails Chemistry; Holland alone fails Physics. Paul and Simu share the top Chemistry score, and Simu leads Physics with 90.

Reference note

The phrase “best score for both subjects” is ambiguous. This solution reports each subject separately; define a combined-score rule if required.

Lab deliverable

Submit the grade distribution, pass/fail vector, student summary, and subject-level failure and top-score analysis.

Factors, Matrices, and Arrays

Chapter at a glance

Before you start: You should be comfortable with vectors, lists, indexing, and arithmetic.

By the end, you can: create and inspect factors, matrices, and arrays; index their dimensions; perform element-wise arithmetic; and use `apply()`.

4.1 Factors

Convert a character vector of regions to a factor:

```
data <- c(
  "East", "West", "East", "North", "North",
  "East", "West", "West", "West", "East", "North"
)

print(data)
print(is.factor(data))

factor_data <- factor(data)
print(factor_data)
print(is.factor(factor_data))
length(factor_data)
```

`is.factor()` confirms the conversion.

4.1.1 Accessing and modifying factor values

```
data <- factor(c(
  "East", "West", "East", "North", "North",
  "East", "West", "West", "West", "East", "North"
))

data[3]
data[3] <- "NorthWest"
```

Assigning a value outside the existing levels produces NA with a warning. Add the level first if it is valid.

4.1.2 Changing level order

```
data <- c(
  "East", "West", "East", "North", "North",
  "East", "West", "West", "West", "East", "North"
)

factor_data <- factor(data)
new_order_data <- factor(
  factor_data,
  levels = c("East", "West", "North")
)
print(new_order_data)
```

The `levels` argument controls category order independently of observation order.

4.1.3 Generating factor levels

```
v <- gl(3, 4, labels = c("Tampa", "Seattle", "Boston"))
print(v)
```

`gl()` generates repeated factor levels.

4.2 Matrices

A matrix stores one data type in rows and columns. `byrow` controls the fill direction:

```
M <- matrix(c(3:14), nrow = 4, byrow = TRUE)
print(M)

N <- matrix(c(3:14), nrow = 4, byrow = FALSE)
print(N)
```

4.2.1 Naming rows and columns

```
rownames <- c("row1", "row2", "row3", "row4")
colnames <- c("col1", "col2", "col3")

P <- matrix(
  c(3:14),
  nrow = 4,
  byrow = TRUE,
  dimnames = list(rownames, colnames)
)
print(P)
```

Dimension names improve readability and indexing.

4.2.2 Matrix indexing

```
print(P[1, 3]) # row 1, column 3
print(P[4, 2]) # row 4, column 2
print(P[2, ])  # all columns from row 2
print(P[, 3])  # all rows from column 3
```

Matrix indices follow `[row, column]`.

4.3 Element-wise matrix arithmetic

Ordinary operators act element by element on conformable matrices:

```
matrix1 <- matrix(c(3, 9, -1, 4, 2, 6), nrow = 2)
matrix2 <- matrix(c(5, 2, 0, 9, 3, 4), nrow = 2)

print(matrix1 + matrix2)
print(matrix1 - matrix2)
print(matrix1 * matrix2)
print(matrix1 / matrix2)
```

Key idea

`*` performs element-wise multiplication; true matrix multiplication uses `%*%`.

4.4 Arrays

Arrays extend matrices to additional dimensions.

4.4.1 Constructing an array

```
vector1 <- c(5, 9, 3)
vector2 <- c(10, 11, 12, 13, 14, 15)

result <- array(
  c(vector1, vector2),
  dim = c(3, 3, 2)
)
print(result)
```

The dimensions specify 3 rows, 3 columns, and 2 layers. Because only 9 values are supplied for 18 positions, R recycles them once.

4.4.2 Naming array dimensions

```
column.names <- c("COL1", "COL2", "COL3")
row.names <- c("ROW1", "ROW2", "ROW3")
matrix.names <- c("Matrix1", "Matrix2")

result <- array(
  c(vector1, vector2),
  dim = c(3, 3, 2),
  dimnames = list(row.names, column.names, matrix.names)
)
print(result)
```

4.4.3 Accessing array elements and slices

```
print(result[3, , 2]) # third row of second matrix
print(result[1, 3, 1]) # row 1, column 3, first matrix
print(result[, , 2]) # entire second matrix
```

Array indices follow [row, column, layer].

4.5 Array manipulation and apply()

apply() runs a function across selected dimensions:

```
new.array <- array(
  c(vector1, vector2),
  dim = c(3, 3, 2)
)

result <- apply(new.array, c(1), sum)
print(result)
```

Margin 1 calculates one sum per row across columns and layers.

Technical note

Lab 4a defines vector3 and vector4 but constructs array2 from the earlier vectors. Confirm the intended inputs before reproducing that step.

Check your understanding

1. Which function converts the region vector into a factor?
2. What does `byrow = TRUE` change when constructing a matrix?
3. In `P[4,2]`, which coordinate is the row and which is the column?
4. How many dimensions are specified by `dim = c(3,3,2)`?
5. What does `result[,2]` select from a three-dimensional array?

Chapter recap

- Factors store categorical values through defined levels.
- Matrices and arrays use dimension-based indexing and may carry names.
- Ordinary arithmetic is element-wise; `apply()` summarizes selected dimensions.

PRACTICAL MILESTONE

Lab 4 of 9

Factors, matrix operations, and multidimensional arrays

Lab 4b guided practice

Coverage. Chapter 4.

Goal. Summarize factor levels, compare a matrix with its transpose, and construct multidimensional arrays.

Task 1 — Retirement-age factor levels.

```
age <- c(
  55, 57, 56, 52, 51, 59, 58, 53, 59, 55,
  60, 60, 60, 60, 52, 55, 56, 51, 60, 52,
  54, 56, 52, 57, 54, 56, 58, 53, 53, 50,
  55, 51, 57, 60, 57, 55, 51, 50, 57, 58
)

age_factor <- factor(age)
print(levels(age_factor))
print(table(age_factor))
```

The exact-age counts are:

Age	50	51	52	53	54	55	56	57	58	59	60
Count	2	4	4	3	2	5	4	5	3	2	6

The requested ranges overlap at 52, 54, 56, and 58. Direct inclusive counts are:

```
range_counts <- c(
  `50-52` = sum(age >= 50 & age <= 52),
  `52-54` = sum(age >= 52 & age <= 54),
  `54-56` = sum(age >= 54 & age <= 56),
  `56-58` = sum(age >= 56 & age <= 58),
  `58-60` = sum(age >= 58 & age <= 60)
)
print(range_counts)
```

The counts are 10, 9, 11, 12, and 11. Do not total them because the groups overlap.

Technical note

Non-overlapping groups require an explicit boundary convention; the source does not provide one.

Task 2 — Create a 3-by-3 matrix and transpose it.

```
V1 <- c(2, 3, 1, 5, 4, 6, 8, 7, 9)

Matrix1 <- matrix(V1, nrow = 3, byrow = TRUE)
Matrix2 <- t(Matrix1)

rownames(Matrix1) <- c("R1", "R2", "R3")
colnames(Matrix1) <- c("C1", "C2", "C3")
rownames(Matrix2) <- c("R1", "R2", "R3")
colnames(Matrix2) <- c("C1", "C2", "C3")
```

```
print(Matrix1 + Matrix2)
print(Matrix1 - Matrix2)
print(Matrix1 * Matrix2)
print(Matrix1 / Matrix2)
```

These operators are element-wise.

Task 3 — Construct the two arrays shown in the expected output.

```
Array1 <- array(1:24, dim = c(2, 4, 3))
Array2 <- array(25:54, dim = c(3, 2, 5))

print(Array1)
print(Array2)

print("The second row of the second matrix of the first array:")
print(Array1[2, , 2])

print("The element corresponding to sample output 30:")
print(Array2[3, 2, 1])
```

Technical note

Array2 has only two columns, although the task requests column 3. The expected value 30 corresponds to `Array2[3, 2, 1]`.

Lab deliverable

Submit the factor summary, range interpretation, matrix results, arrays, and requested selections.

Stage 3

Programming Flow and Data Frames

Control Flow

Chapter at a glance

Before you start: You should be able to construct logical expressions.

By the end, you can: use conditional statements, `switch()`, loops, `break`, and `next`.

5.1 Selection statements

Conditional statements select which code block runs.

5.1.1 Simple `if`

```
x <- 30L

if (is.integer(x)) {
  print("X is an Integer")
}
```

The body runs only when the condition is TRUE.

5.1.2 `if...else`

```
x <- c("what", "is", "truth")

if ("Truth" %in% x) {
  print("Truth is found")
} else {
  print("Truth is not found")
}
```

Character comparison is case-sensitive: "Truth" and "truth" differ.

5.1.3 `if...else if...else`

```
x <- c("what", "is", "truth")

if ("Truth" %in% x) {
  print("Truth is found the first time")
} else if ("truth" %in% x) {
  print("truth is found the second time")
} else {
  print("No truth found")
}
```

Conditions are checked in order until one succeeds.

5.2 `switch()`

With a numeric selector, `switch()` returns the matching position:

```
x <- switch(
  3,
  "first",
  "second",
  "third",
  "fourth"
)
print(x)
```

Selector 3 returns "third".

5.3 Repetition statements

R provides repeat, while, and for loops.

5.3.1 repeat loop

```
v <- c("Hello", "loop")
cnt <- 2

repeat {
  print(v)
  cnt <- cnt + 1

  if (cnt > 5) {
    break
  }
}
```

repeat requires an internal exit such as break.

5.3.2 while loop

```
v <- c("Hello", "while loop")
cnt <- 2

while (cnt < 7) {
  print(v)
  cnt = cnt + 1
}
```

while checks its condition before each iteration; the update prevents an infinite loop.

5.3.3 for loop

for iterates over vectors or lists:

```
v <- LETTERS[1:4]
for (i in v) {
  print(i)
}

for (x in 1:10) {
  print(x)
}

fruits <- list("apple", "banana", "cherry")
for (x in fruits) {
  print(x)
}
```

5.4 Controlling a loop with `break` and `next`

`break` exits the current loop:

```
fruits <- list("apple", "banana", "cherry")

for (x in fruits) {
  if (x == "cherry") {
    break
  }
  print(x)
}
```

`next` skips the rest of the current iteration:

```
for (x in fruits) {
  if (x == "banana") {
    next
  }
  print(x)
}
```

5.5 Conditions inside loops

The dice example combines iteration, a comparison, and an `if...else` block:

```
dice <- 1:6

for (x in dice) {
  if (x == 6) {
    print(paste("The dice number is", x, "Yahtzee!"))
  } else {
    print(paste("The dice number is", x, "Not Yahtzee"))
  }
}
```

This pattern iterates, tests, and produces a conditional result.

5.6 Nested loops

The final Lab 5a example places one `for` loop inside another:

```
adj <- list("red", "big", "tasty")
fruits <- list("apple", "banana", "cherry")

for (x in adj) {
  for (y in fruits) {
    print(paste(x, y))
  }
}
```

The nested loops generate every adjective–fruit pair.

Check your understanding

1. What must be TRUE for the body of a simple `if` statement to run?
2. Why does the first branch of the handout's "Truth" example fail while the second succeeds?
3. How does a repeat loop terminate in the supplied pattern?
4. What is the difference between `break` and `next` in the fruit examples?

5. How many adjective-fruit combinations are generated by three adjectives and three fruits?

Chapter recap

- Conditionals and `switch()` select code paths.
- `repeat`, `while`, and `for` express different repetition patterns.
- `break` exits a loop; `next` skips one iteration.

PRACTICAL MILESTONE

Lab 5 of 9

Leap years, cubes, and Armstrong numbers

Lab 5b guided practice

Coverage. Chapter 5.

Goal. Solve three problems with conditions and loops.

Task 1 — Check whether a year is a leap year.

```
year <- as.integer(readline("Input year: "))

if ((year %% 400 == 0) ||
    (year %% 4 == 0 && year %% 100 != 0)) {
  print(paste(year, "is a leap year."))
} else {
  print(paste(year, "is not a leap year."))
}
```

The sample classifies 2004 as a leap year and 1900 as a common year.

Task 2 — Display cubes up to an input integer.

```
n <- as.integer(readline("Input an integer: "))

for (i in seq_len(n)) {
  cube <- i ^ 3
  print(paste(
    "Number is:", i,
    "and cube of the", i, "is:", cube
  ))
}
```

For $n = 5$, the cubes are 1, 8, 27, 64, and 125.

Task 3 — Check an Armstrong number.

An Armstrong number equals the sum of its digits raised to the number of digits. This definition correctly classifies the supplied example 1634.

```
number <- as.integer(readline("Input an integer: "))

original <- number
digits <- nchar(as.character(number))
sum_power <- 0

temp <- number
while (temp > 0) {
  digit <- temp %% 10
  sum_power <- sum_power + digit ^ digits
  temp <- temp %% 10
}

if (sum_power == original) {
  print(paste(original, "is an Armstrong number."))
} else {
  print(paste(original, "is not an Armstrong number."))
}
```

Technical note

The source also says “power three,” which conflicts with its four-digit example. Confirm the required definition with your instructor.

Lab deliverable

Submit all three programs with representative test results.

Data Frame Fundamentals

Chapter at a glance

Before you start: You should understand vectors, lists, indexing, and basic control flow.

By the end, you can: create, inspect, index, extend, and reduce data frames.

6.1 Creating a data frame

Create an employee table from equal-length vectors:

```
emp.data <- data.frame(  
  emp_id = 1:5,  
  emp_name = c("Rick", "Dan", "Michelle", "Ryan", "Gary"),  
  salary = c(623.3, 515.2, 611.0, 729.0, 843.25)  
)
```

A data frame stores named, equal-length columns that may have different types.

6.2 Creating a data frame from another structure

6.2.1 From a matrix

```
my_matrix <- matrix(c(1, 2, 3, 4, 5, 6), nrow = 2)  
print(my_matrix)  
  
df_from_matrix <- data.frame(my_matrix)  
print(df_from_matrix)  
  
names(df_from_matrix) <- c("col_1", "col_2", "col_3")  
print(df_from_matrix)
```

The generated column names are then replaced.

6.2.2 From a list of vectors

```
my_list <- list(  
  rating = 1:4,  
  animal = c("koala", "hedgehog", "sloth", "panda"),  
  country = c("Australia", "Italy", "Peru", "China"),  
  avg_sleep_hours = c(21, 18, 17, 10)  
)  
  
print(my_list)  
super_sleepers <- data.frame(my_list)  
print(super_sleepers)
```

The equal-length list components become columns.

6.3 Displaying and inspecting a data frame

Print the table directly:

```
print(emp.data)
```

RStudio's viewer displays the same object in a spreadsheet-like window.

6.3.1 Structure

```
str(emp.data)
```

`str()` reports columns and stored types.

6.3.2 Summary and dimensions

Use summary and dimension functions for different purposes:

```
print(summary(emp.data))  
  
print(dim(emp.data))  
print(ncol(emp.data))  
print(nrow(emp.data))
```

`summary()` describes columns; `dim()`, `ncol()`, and `nrow()` report shape.

Technical note

Lab 6a appears to swap the headings for summary and dimension output. The functions above are organized by their actual behavior.

6.4 Accessing columns and rows

The salary column is extracted in three forms:

```
print(emp.data$salary)  
print(emp.data[["salary"]])  
print(emp.data[3])
```

`$` and `[[` return a column vector; `[3]` returns a one-column data frame.

The first two rows with all columns are selected using:

```
result <- emp.data[1:2, ]  
print(result)
```

Data frames use `[row, column]` indexing.

6.5 Adding columns

Add a column by assigning a new name:

```
emp.data$dept <- c("IT", "Operations", "IT", "HR", "Finance")  
print(emp.data)
```

`cbind()` can attach another column:

```
emp.newdata <- cbind(
```

```
emp.data,
start_date = as.Date(c(
  "2012-01-01",
  "2013-09-23",
  "2014-11-15",
  "2014-05-11",
  "2015-03-27"
))
)
```

The resulting table includes character, numeric, and date columns.

6.6 Adding rows

Append compatible rows with `rbind()`:

```
emp.newdata <- data.frame(
  emp_id = c(6:8),
  emp_name = c("Rasmi", "Pranab", "Tusar"),
  salary = c(578.0, 722.5, 632.8),
  dept = c("IT", "Operations", "Finance")
)

emp.finaldata <- rbind(emp.data, emp.newdata)
print(emp.finaldata)
```

The column names and compatible types must match.

6.7 Removing rows and columns

Negative indices exclude rows or columns:

```
# Remove the second row and first column
result <- emp.data[-2, -1]
print(result)
```

6.8 Extra practice table

Extend this practice table with additional rows, columns, and types:

```
friends <- data.frame(
  friend_id = 1:5,
  friend_name = c("Sachin", "Sourav", "Dravid", "Sehwag", "Dhoni"),
  location = c("Kolkata", "Delhi", "Bangalore", "Hyderabad", "Chennai")
)
print(friends)
```

Check your understanding

1. What does `str(emp.data)` help you inspect?
2. Which functions in the handout report the number of rows and columns?
3. Give two supplied ways to access the salary column by name.
4. Which function binds new columns and which function binds new rows?
5. How is negative indexing reused to remove data-frame rows or columns?

Chapter recap

- Data frames combine equal-length columns of potentially different types.
- Inspect structure, summaries, and dimensions with dedicated functions.
- Index with \$, brackets, or row/column coordinates; extend with `cbind()` and `rbind()`.

PRACTICAL MILESTONE

Lab 6 of 9

Build and extend a student data frame

Lab 6 guided practice

Coverage. Chapter 6.

Goal. Build and extend a student table, then inspect its structure and summaries.

Task 1 — Create the base data frame.

```
students <- data.frame(  
  name = c(  
    "Anastasia", "Dima", "Michael", "Matthew",  
    "Laura", "Kevin", "Jonas"  
  ),  
  score = c(12.5, 9.0, 16.5, 12.0, 9.0, 8.0, 19.0),  
  attempts = c(1, 3, 2, 3, 2, 1, 2)  
)  
  
print(students)
```

Task 2 — Add the specified qualify column.

```
students$qualify <- c(  
  "yes", "no", "yes", "no", "no", "no", "yes"  
)  
print(students)
```

Task 3 — Add Emily's row.

```
emily <- data.frame(  
  name = "Emily",  
  score = 14.5,  
  attempts = 1,  
  qualify = "yes"  
)  
  
students <- rbind(students, emily)  
print(students)
```

Task 4 — Inspect the dataset.

```
str(students)  
summary(students)  
dim(students)  
nrow(students)  
ncol(students)
```

Convert qualify to a factor to see how its summary changes:

```
students$qualify <- factor(students$qualify)  
summary(students)
```

The final table has 8 rows, 4 columns, and scores from 8.0 to 19.0.

Lab deliverable

Submit the table before and after modification, its structure and summaries, dimensions, and a brief note on the factor conversion.

Managing Data Frames

Chapter at a glance

Before you start: You should be able to create, inspect, and index data frames.

By the end, you can: import, inspect, clean, rename, filter, sort, and export tabular data.

7.1 Packages used in Lab 7a

Lab 7 uses three packages:

Package	Purpose
readxl	Import the Excel file <code>uforeport.xls</code> .
dplyr	Filter, pipe, and arrange data frames.
xlsx	Export an Excel file.

Install packages once; load them in each session:

```
install.packages("readxl")
library(readxl)

any(grepl("readxl", installed.packages()))
```

The final line checks whether `readxl` appears among installed packages.

7.2 Working directory and file placement

Keep source files in the RStudio project and confirm the active directory:

```
getwd()
list.files()
```

Place `uforeport.xls` and `titanic.csv` in that project directory.

Key idea

If import fails, check `getwd()` and `list.files()` before changing the analysis.

7.3 Importing Excel and CSV files

Import Excel with `read_excel()`:

```
uforeport <- read_excel("uforeport.xls")
View(uforeport)
```

Import CSV with base R:


```
titanic <- read.csv("titanic.csv")
View(titanic)
```

7.4 Identifying missing values

`is.na()` identifies missing values.

7.4.1 Count all missing values

```
sum(is.na(uforeport))
sum(is.na(titanic))
```

7.4.2 Find their positions

```
which(is.na(uforeport))
which(is.na(titanic))
```

A small vector illustrates both operations:

```
demo <- c(1, 2, NA, 4, NA, 6, 7)

sum(is.na(demo))
which(is.na(demo))
```

7.4.3 Count missing values by column

```
print(sapply(uforeport, function(x) sum(is.na(x))))
print(sapply(titanic, function(x) sum(is.na(x))))
```

This returns one missing-value count per column.

7.5 Removing rows with missing values

Compare dimensions before and after removing incomplete rows:

```
dim(uforeport)
uforeport_cleaned <- na.omit(uforeport)
dim(uforeport_cleaned)

dim(titanic)
titanic_cleaned <- na.omit(titanic)
dim(titanic_cleaned)
```

The difference shows how many rows were removed.

Technical note

`na.omit()` removes every row containing at least one missing value. Confirm that this loss is acceptable before using the cleaned data for analysis.

7.6 Column names and renaming

`names()` and `colnames()` both retrieve data-frame column names:

```
colnames(uforeport_cleaned)
colnames(titanic_cleaned)

names(uforeport_cleaned)
names(titanic_cleaned)
```

Replace spaces with underscores before filtering:

```
names(uforeport_cleaned) <- gsub(
  " ", "_", colnames(uforeport_cleaned)
)
```

For example, Colors Reported becomes Colors_Reported.

7.7 Filtering data

Use `dplyr::filter()` for exact and numeric conditions:

```
install.packages("dplyr")
library(dplyr)

print(filter(uforeport_cleaned, City == "Belton"))
print(filter(uforeport_cleaned, Colors_Reported == "GREEN"))
print(filter(titanic_cleaned, sex == "female"))
print(filter(titanic_cleaned, fare > 500))
```

7.7.1 Multiple conditions

Multiple conditions may be written explicitly or separated inside `filter()`:

```
print(filter(
  titanic_cleaned,
  sex == "female" & fare > 500
))

# Alternative dplyr form
titanic_cleaned %>%
  filter(sex == "female", fare > 500)

ufo_green <- uforeport_cleaned %>%
  filter(Colors_Reported == "GREEN")
```

7.8 Arranging rows

Sort rows with `arrange()`:

```
# Ascending
titanic_sortbyfare <- arrange(titanic_cleaned, fare)

# Descending
titanic_sortbyfare <- arrange(titanic_cleaned, desc(fare))
```

Because the variable is reassigned, only the descending result remains after both lines.

7.9 Exporting analysis results

Export the transformed results:

```
install.packages("xlsx")
library(xlsx)
```

```
write.xlsx(ufo_green, "ufo_green.xlsx")

write.csv(
  titanic_sortbyfare,
  "titanic_sortbyfare.csv"
)
```

The complete workflow is **import** → **inspect** → **clean** → **transform** → **export**.

Check your understanding

1. Which function tells you the current working directory?
2. Which import function is used for `ufo_report.xls`, and which for `titanic.csv`?
3. How do `sum(is.na(x))` and `which(is.na(x))` answer different questions?
4. What evidence does the handout collect before and after `na.omit()`?
5. Which `dplyr` function sorts rows, and how is descending order requested?

Chapter recap

- Project-relative files make imports reproducible.
- Inspect missingness before removing incomplete rows.
- Rename, filter, sort, and export only after confirming the data structure.

PRACTICAL MILESTONE

Lab 7 of 9

Clean, filter, and report on external datasets

Lab 7b analysis workflow

Coverage. Chapter 7.

Required files. `uforeport.xls` and `titanic.csv`.

Goal. Clean both datasets and report reproducible observations.

Technical note

The required datasets were not included in this notes build, so no numerical findings are asserted here. Verify every result against the files supplied in class.

Technical note

Check what the label column actually is before you interpret it. The Titanic file used in the lab folder (`tested.csv`) comes from the Kaggle *test* set, where `Survived` is a placeholder in which every female is 1 and every male is 0. One line of code proves it:

```
all((titanic$Sex == "female") == (titanic$Survived == 1))
# TRUE -> the column carries no independent information
```

Reporting “100% of women survived” from that file is reading the labelling rule back out, not discovering anything about 1912. Use `train.csv`, which holds the real outcomes, before making any survival claim. Run this identity check on any dataset whose target column you did not create yourself.

Part A — Import and clean.

```
library(readxl)
library(dplyr)

uforeport <- read_excel("uforeport.xls")
titanic <- read_csv("titanic.csv")

print(sapply(uforeport, function(x) sum(is.na(x))))
print(sapply(titanic, function(x) sum(is.na(x))))

uforeport_cleaned <- na.omit(uforeport)
titanic_cleaned <- na.omit(titanic)

names(uforeport_cleaned) <- gsub(
  " ", "_", names(uforeport_cleaned)
)
```

Part B — Build an evidence table before writing conclusions.

Inspect the imported schema and calculate counts before percentages:

```
# Confirm actual column names first.
print(names(titanic_cleaned))

# Example patterns; adapt to the exact imported schema.
print(table(titanic_cleaned$sex))
print(table(titanic_cleaned$survived))
print(table(titanic_cleaned$embark_town))
```

```
# Cross-tabulation pattern
print(table(
  titanic_cleaned$embark_town,
  titanic_cleaned$survived
))
```

Part C — Filter targeted subsets.

```
female_rows <- titanic_cleaned %>%
  filter(sex == "female")

high_fare_rows <- titanic_cleaned %>%
  filter(fare > 500)

green_ufo <- uforeport_cleaned %>%
  filter(Colors_Reported == "GREEN")
```

Part D — Arrange and export supporting tables.

```
titanic_fare_desc <- titanic_cleaned %>%
  arrange(desc(fare))

write.csv(
  titanic_fare_desc,
  "titanic_fare_desc.csv"
)
```

Report structure.

1. State the original and cleaned dimensions of both datasets.
2. Report which columns contained missing values and how many rows were removed by `na.omit()`.
3. Present filtered or grouped observations supported by printed counts.
4. Distinguish a raw count from a percentage and show the denominator used for each percentage.
5. Export at least one useful filtered/sorted result as evidence of the data-management workflow.

Lab deliverable

Submit a cleaned-data report for both files. Every numerical claim must be reproducible from the submitted code.

Stage 4

Visualization and Introductory Analysis

Data Visualization in R

Chapter at a glance

Before you start: You should be able to load and inspect data frames.

By the end, you can: create and interpret pie charts, box plots, histograms, scatterplots, bar charts, and line graphs.

8.1 Using built-in datasets

List built-in datasets, then load and inspect `mtcars`:

```
data()

data(mtcars)
print(head(mtcars, 10))

input <- mtcars[, c("mpg", "cyl")]
print(head(input))
```

The workflow is: load, inspect, select, then plot.

8.2 Built-in graphics demonstrations

```
demo(graphics)
demo(image)
```

Press Enter to advance through each demonstration.

8.3 Pie charts

A basic pie chart uses values and labels:

```
x <- c(21, 62, 10, 53)
labels <- c("London", "New York", "Singapore", "Mumbai")

pie(x, labels)
```

Save the plot by opening a graphics device and closing it afterward:

```
png(file = "city.png")
pie(x, labels)
dev.off()
```

8.3.1 Title, colors, percentages, and legend

```
pie(
  x,
  labels,
  main = "City pie chart",
```

```

col = rainbow(length(x))
)

piepercent <- round(100 * x / sum(x), 1)

pie(
  x,
  labels = piepercent,
  main = "City pie chart",
  col = rainbow(length(x))
)

legend(
  "topright",
  c("London", "New York", "Singapore", "Mumbai"),
  cex = 0.8,
  fill = rainbow(length(x))
)

```

The labels show each value as a percentage of the total.

8.3.2 3D pie chart

The optional `plotrix` example requires defined labels:

```

library(plotrix)

pie3D(
  x,
  labels = labels,
  explode = 0.1,
  main = "Pie Chart of Countries"
)

```

Technical note

The source uses an undefined object, `lb1`; the working example reuses `labels`. Prefer a clear 2D chart when depth adds no information.

8.4 Box plots

Compare mileage across cylinder groups with formula notation:

```

boxplot(
  mpg ~ cyl,
  data = mtcars,
  xlab = "Number of Cylinders",
  ylab = "Miles Per Gallon",
  main = "Mileage Data"
)

```

A more customized version adds notches, variable widths, colors, and supplied group names:

```

boxplot(
  mpg ~ cyl,
  data = mtcars,
  xlab = "Number of Cylinders",
  ylab = "Miles Per Gallon",
  main = "Mileage Data",
  notch = TRUE,
  varwidth = TRUE,
  col = c("green", "yellow", "purple"),
  names = c("4 cylinders", "6 cylinders", "8 cylinders")
)

```


8.5 Histograms

```
v <- c(9, 13, 21, 8, 36, 22, 12, 41, 31, 33, 19)

hist(
  v,
  xlab = "Weight",
  col = "yellow",
  border = "blue"
)
```

Add axis limits and a requested break count:

```
hist(
  v,
  xlab = "Weight",
  col = "green",
  border = "red",
  xlim = c(0, 50),
  ylim = c(0, 5),
  breaks = 5
)
```

Technical note

The source prints the invalid argument `n breaks = 5`; the correct argument is `breaks = 5`.

8.6 Scatterplots

Lab 8a selects weight and mileage from `mtcars`:

```
input <- mtcars[, c("wt", "mpg")]

plot(
  x = input$wt,
  y = input$mpg,
  xlab = "Weight",
  ylab = "Mileage",
  xlim = c(2.5, 5),
  ylim = c(15, 30),
  main = "Weight versus mileage"
)
```

A scatterplot matrix expands the idea to four variables:

```
pairs(
  ~ wt + mpg + disp + cyl,
  data = mtcars,
  main = "Scatterplot Matrix"
)
```

8.7 Bar charts

A basic bar chart is:

```
H <- c(7, 12, 28, 3, 41)
barplot(H)
```

The customized version supplies month names, labels, title, fill color, and border color:

```
H <- c(7, 12, 28, 3, 41)
M <- c("Mar", "Apr", "May", "Jun", "Jul")
```

```
barplot(
  H,
  names.arg = M,
  xlab = "Month",
  ylab = "Revenue",
  col = "blue",
  main = "Revenue chart",
  border = "red"
)
```

8.7.1 Stacked matrix input

Lab 8a constructs a 3-by-5 matrix of values and passes it to `barplot()` with a region legend:

```
colors <- c("green", "orange", "brown")
months <- c("Mar", "Apr", "May", "Jun", "Jul")
regions <- c("East", "West", "North")

Values <- matrix(
  c(2,9,3,11,9, 4,8,7,3,12, 5,2,8,10,11),
  nrow = 3,
  ncol = 5,
  byrow = TRUE
)

barplot(
  Values,
  main = "Total revenue",
  names.arg = months,
  xlab = "Month",
  ylab = "Revenue",
  col = colors
)

legend("topleft", regions, cex = 1.3, fill = colors)
```

8.8 Line graphs

```
v <- c(7, 12, 28, 3, 41)
plot(v, type = "o")
```

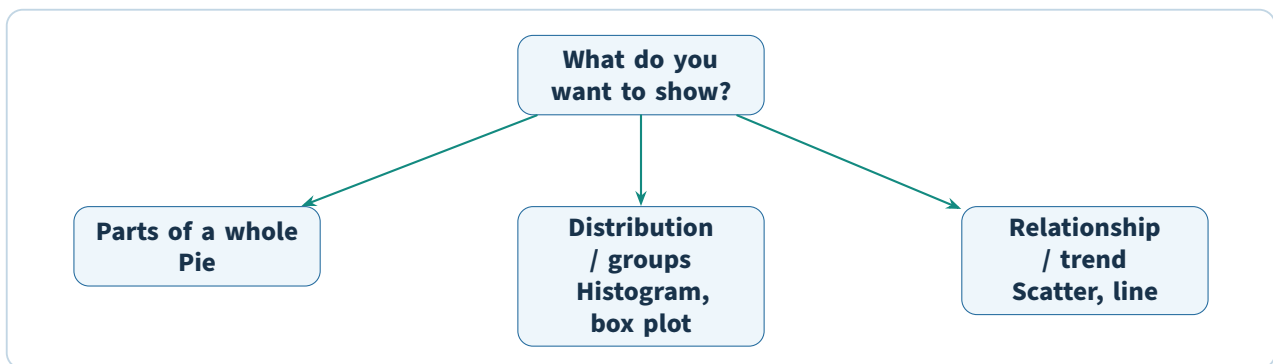
Add a title, labels, and color:

```
plot(
  v,
  type = "o",
  col = "red",
  xlab = "Month",
  ylab = "Rainfall",
  main = "Rainfall chart"
)
```

A second series is added using `lines()`:

```
t <- c(14, 7, 6, 19, 3)

plot(
  v,
  type = "o",
  col = "red",
  xlab = "Month",
  ylab = "Rainfall",
  main = "Rainfall chart"
)
lines(t, type = "o", col = "blue")
```



Choose a chart that matches the analytical question, then explain the visible evidence.

Check your understanding

1. Which command lists built-in datasets and which command loads `mtcars`?
2. How are percentages calculated for the pie chart in the handout?
3. Which plot compares `mpg` across `cyl` groups?
4. What does `pairs()` produce in the supplied example?
5. Which function adds a second series to an existing line plot?

Chapter recap

- Match the chart type to composition, distribution, comparison, relationship, or trend.
- Use titles, labels, legends, and colors to clarify the evidence.
- Every visualization should support a specific analytical claim.

PRACTICAL MILESTONE

Lab 8 of 9

Turn data-management results into visual evidence

Lab 8b visualization workflow

Coverage. Chapter 8 and the cleaned-data workflow from Lab 7.

Goal. Add two useful plots to Lab 7 and present a concise story from a built-in dataset.

Part A — Add visual evidence to the external-file report.

Use the cleaned Lab 7 objects and choose two plot types that match the verified columns.

Example patterns:

```
# Example: categorical count bar chart
sex_counts <- table(titanic_cleaned$sex)
barplot(
  sex_counts,
  main = "Passenger count by sex",
  xlab = "Sex",
  ylab = "Count"
)

# Example: numeric distribution
hist(
  titanic_cleaned$fare,
  main = "Fare distribution",
  xlab = "Fare"
)
```

Base each claim on the computed counts or visible plot.

Part B — Storytelling with a built-in dataset.

Any built-in dataset is acceptable. This example uses `mtcars`:

```
data(mtcars)

boxplot(
  mpg ~ cyl,
  data = mtcars,
  xlab = "Number of cylinders",
  ylab = "Miles per gallon",
  main = "Mileage by cylinder group"
)

plot(
  mtcars$wt,
  mtcars$mpg,
  xlab = "Weight",
  ylab = "Miles per gallon",
  main = "Weight versus mileage"
)
```

Structure the discussion as follows:

1. Identify what each axis or group represents.
2. Describe the strongest pattern without claiming causation.
3. Connect the plot to a practical question a manager or analyst might ask.
4. If a second plot supports the same observation, explain how the evidence complements the first plot.

Lab deliverable

Submit the revised Lab 7 report with two plot types and a separate evidence-based story using a built-in dataset.

Correlation, Transformation, and Scaling

Chapter at a glance

Before you start: You should be comfortable loading data, selecting columns, using packages, and creating plots.

By the end, you can: compute and test Pearson correlation, visualize correlation matrices, and compare log transformation, standardization, and min-max scaling.

9.1 Pearson correlation with `cor()`

Compute a Pearson coefficient for two paired vectors:

```
x <- c(1, 2, 3, 4, 5, 6, 7)
y <- c(1, 3, 6, 2, 7, 4, 5)

result <- cor(x, y, method = "pearson")
cat("Pearson correlation coefficient is:", result)
```

The coefficient summarizes their linear association from -1 to 1.

9.2 Pearson correlation with `cor.test()`

`cor.test()` adds inferential output:

```
result <- cor.test(x, y, method = "pearson")
print(result)
```

Its output includes the estimate, test statistic, p-value, and confidence interval.

Technical note

Correlation measures association, not causation. Inspect the data and assumptions before interpreting the coefficient or p-value.

9.3 Correlation matrix with `corrplot`

Compute and visualize the `mtcars` correlation matrix:

```
install.packages("corrplot")
install.packages("RColorBrewer")

library(corrplot)
library(RColorBrewer)

M <- cor(mtcars)
?corrplot

corrplot(M, type = "upper")
corrplot(M, type = "upper", order = "hclust")
corrplot(
  M,
```

```

type = "upper",
order = "hclust",
col = brewer.pal(n = 8, name = "RdYlBu")
)

```

Hierarchical ordering places variables with similar correlation patterns together.

9.4 Building a heatmap-ready correlation table

Use the environmental dataset to prepare a clustered heatmap:

```

install.packages("lattice")
library(lattice)

data <- environmental
head(data)

corr_mat <- round(cor(data), 2)
head(corr_mat)

```

The matrix is reordered using a distance derived from correlation values and hierarchical clustering:

```

corr_dist <- as.dist((1 - corr_mat) / 2)
hc <- hclust(corr_dist)
corr_mat <- corr_mat[hc$order, hc$order]

```

Convert the matrix to long form for ggplot2:

```

install.packages("reshape2")
library(reshape2)

melted_corr_mat <- melt(corr_mat)
head(melted_corr_mat)

```

9.5 Correlation heatmaps

9.5.1 Using ggplot2

```

install.packages("ggplot2")
library(ggplot2)

ggplot(
  data = melted_corr_mat,
  aes(x = Var1, y = Var2, fill = value)
) +
  geom_tile() +
  geom_text(
    aes(label = value),
    color = "white",
    size = 4
  )

```

Tile color and overlaid text both encode the correlation value.

9.5.2 Using heatmaply

```

install.packages("heatmaply")
library(heatmaply)

heatmaply_cor(
  x = cor(data),
  xlab = "Features",
  ylab = "Features",

```

```
k_col = 2,  
k_row = 2  
)
```

9.5.3 Using ggcorrplot

Use the correctly named package:

```
install.packages("ggcorrplot")  
library(ggcorrplot)  
ggcorrplot::ggcorrplot(cor(data))
```

Technical note

The source misspells the installation name as ggcorplot. The package and namespace are ggcorrplot.

9.6 Log transformation

The supplied numeric vector is:

```
mydata <- c(244, 753, 596, 645, 874, 141, 639, 465, 999, 654)  
  
scaled_data <- log(mydata)  
print(scaled_data)
```

Log transformation compresses large values and can reduce right skew; it does not constrain values to a fixed range.

9.7 Standardization

scale() centers values at approximately zero and scales them to unit standard deviation:

```
scaled_data2 <- as.data.frame(scale(mydata))  
print(scaled_data2)
```

9.8 Min-max scaling

Min-max scaling maps values to a fixed range, normally 0 to 1:

```
install.packages("caret")  
library(caret)  
  
minmax <- preProcess(  
  as.data.frame(mydata),  
  method = c("range")  
)  
  
scaled_data3 <- predict(  
  minmax,  
  as.data.frame(mydata)  
)  
print(scaled_data3)
```

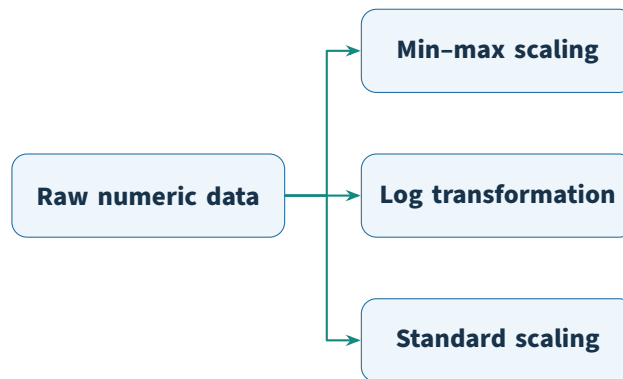

9.9 Comparing summaries

Compare the raw and transformed summaries:

```
summary(mydata)
summary(scaled_data)
summary(scaled_data2)
summary(scaled_data3)
```

Technical note

The source calls an undefined object, `scaled_data1`. The corrected comparison uses `scaled_data`.



Compare each method with the raw data because the resulting scales have different interpretations.

Check your understanding

1. What is the difference in output intent between `cor()` and `cor.test()` in Lab 9a?
2. Which dataset is used for the first `corrplot()` example?
3. Why is `melt()` applied to the reordered correlation matrix before the `ggplot2` heatmap?
4. Which function performs the standard scaling in the handout?
5. Which object-name mismatch appears in the final summary comparison?

Chapter recap

- `cor()` estimates association; `cor.test()` adds inferential statistics.
- Clustered heatmaps group variables with similar correlation patterns.
- Log transformation, standardization, and min-max scaling serve different purposes.

PRACTICAL MILESTONE

Lab 9 of 9

Correlation heatmaps and normalization comparison

Lab 9b guided workflow

Coverage. Chapter 9.

Goal. Analyze ToothGrowth correlations and compare three transformations of mtcars.

Task 1 — ToothGrowth: correlation and heatmap.

Inspect the dataset and select its numeric columns before calculating correlation.

```
data(ToothGrowth)
str(ToothGrowth)

numeric_tooth <- ToothGrowth[, sapply(ToothGrowth, is.numeric)]

corr_tooth <- cor(numeric_tooth)
print(corr_tooth)

# One heatmap option taught in Lab 9a
install.packages("corrplot")
library(corrplot)
corrplot(corr_tooth, type = "upper")
```

Interpret coefficients, not color alone. Note that excluding the categorical supp variable means this simple correlation does not adjust for supplement type.

Task 2 — mtcars: transformation and scaling.

Inspect the data before applying the three requested methods:

```
data(mtcars)
str(mtcars)
summary(mtcars)
```

Use `log1p()` because `mtcars` contains zeros:

```
# Log transformation that handles zero values
mtcars_log <- log1p(mtcars)

# Standard scaling
mtcars_standard <- as.data.frame(scale(mtcars))

# Min-max scaling
install.packages("caret")
library(caret)

minmax_model <- preProcess(mtcars, method = c("range"))
mtcars_minmax <- predict(minmax_model, mtcars)
```

Technical note

`log1p(x)` computes $\log(1 + x)$, so it remains finite at zero. State this choice because it differs from applying `log()` directly.

Compare the representations using the same variables and the same descriptive functions:

```
summary(mtcars)
summary(mtcars_log)
summary(mtcars_standard)
summary(mtcars_minmax)
```

A concise comparison should include:

- the range/center of selected raw columns;
- the corresponding summary after log transformation;
- the same columns after standard scaling;
- the same columns after min–max scaling;
- any non-finite values or other issues encountered during transformation.

Lab deliverable

Submit the `ToothGrowth` matrix and heatmap with a brief interpretation, plus an `mtcars` comparison of all three methods and stated implementation choices.

Stage 5

Capstone

Capstone: Cleaning a Real Messy Dataset

Every earlier lab handed you data that was already tidy. This one does not. `Unclean Dataset.csv` is a student enrolment file that has been through several careless hands, and the task is to turn it into something a database could accept — then say honestly what the cleaning cost.

Source files. `Lab Final Project/Lab Final Project.R`, `Unclean Dataset.csv` (136 data rows), and `Cleaned Dataset.csv` (the expected 64-row result).

Goal. Repair a file with mixed delimiters, shifted columns, inconsistent categories, three date formats, and currency text, and report what was discarded.

Lab guidance

Run it from the folder that holds it, so the relative paths resolve:

```
setwd("path/to/DATA SCIENCE/Lab Final Project")
source("Lab Final Project.R")
```

It needs `tidyverse` and `lubridate`. The script checks for them and stops with instructions rather than installing them, because a lab script that silently writes to a shared machine's package library is bad manners.

Seven defects, and what each one teaches

Defect in the file	Why it cannot simply be read with <code>read.csv()</code>
Two delimiters	Some rows use <code> </code> , others use <code>,</code> . One <code>read.csv()</code> call cannot parse both, so the script splits the lines first and parses each group with the right delimiter.
Trailing commas	Pipe rows end in <code>\$1200,,,,,</code> , which would create phantom empty columns.
Shifted columns	In some comma rows the age has slipped into the Gender field, so a value like <code>M24</code> holds two variables at once.
Inconsistent categories	Gender appears as <code>M</code> , <code>Male</code> , <code>male</code> , <code>F</code> , <code>Female</code> ; the script folds case and keys on the first letter.
Misspelt courses	<code>Machine Learnin</code> , <code>Web Development</code> . Repaired by pattern, not by a lookup table, because the truncations vary.
Three date formats	<code>ymd</code> , <code>dmy</code> , and <code>mdy</code> in one column. <code>parse_date_time()</code> is given all three orders.
Currency as text	<code>\$1,200</code> is a string. Strip every non-digit, then convert.

Key idea

Notice the order the script works in. It repairs *structure* first (split the delimiters, unshift the columns), then *values* (categories, dates, currency), and only then drops rows. Reversing that order throws away rows that were merely mis-parsed rather than genuinely bad.

The number that matters most

The script now prints a cleaning summary. On the supplied file it reads:

```
raw data lines read      : 136
pipe-delimited          : 76
```

```
comma-delimited      : 60
rows after merge      : 136
rows after clean/dedupe : 64
dropped              : 72
Dates that failed to parse: 7
```

Technical note

53% of the rows did not survive. A cleaning script that printed only `head(clean_df)` would look like a complete success while discarding more than half the dataset, and nobody reading the tidy output would know.

Before you present the cleaned file, account for those 72 rows. How many were true duplicates removed by `distinct()`? How many failed the `!is.na(Student_ID)` filter, and were those genuinely unusable or just parsed badly? Seven enrolment dates are still NA even in the output you keep.

“Rows dropped” is not a footnote. It is the main finding of a cleaning exercise, and it belongs in the report next to the row count you kept.

Try it yourself

Add one line before the `distinct()` call to count exact duplicates separately from rows removed by the `is.na` filter:

```
cat("exact duplicates:", nrow(full_df) - nrow(distinct(full_df)), "\n")
```

Then inspect the seven unparsed dates and decide whether a fourth format order would rescue them, or whether they are genuinely unreadable:

```
full_df$Enrollment_Date[is.na(clean_df$Enrollment_Date)]
```

Either answer is acceptable in the report. Not looking is not.

Key idea

The script also adds a `System_Record_ID` surrogate key, because `Student_ID` repeats in this file. A natural key that is not actually unique will break the first join you attempt, so a cleaning step that will feed a database should end by giving every row an identifier it controls.

Lab deliverable

Submit the cleaned CSV, the script, and a short data-quality note stating: the original and final row counts, how many rows each cleaning rule removed and why, which fields you standardised and to what convention, how many dates remain unparsed, and one sentence on whether the surviving 64 rows are still representative of the original file.